2.1 函数参数进阶

可变关键字参数

- 概念
  - 定义函数时，在形参名前加两个星号 **，该形参可以接收任意多个关键字参数
  - 传入的关键字参数被组装为一个字典，参数名为键，参数值为值
  - 未传入任何关键字参数时，该形参为空字典
  - 带 ** 的形参必须位于参数列表的最后
- 与可变数量参数 * 的对比：

比较项	`*args`	`**kwargs`
接收的实参	任意多个位置参数	任意多个关键字参数
组装后的类型	元组	字典

可变关键字参数

- 示例：输出学生信息，除姓名外的其他信息项不固定

```
def show_info(name, **info):
    print(info)
    print("姓名: {}".format(name))
    for key, value in info.items():
        print("{}: {}".format(key, value))

show_info("张三", age=20, major="计算机")
show_info("李四")
```

• 第一次调用："张三" 按位置传给 name，age=20 和 major="计算机" 由 info 接收，组装为字典
• 第二次调用：没有传入关键字参数，info 为空字典，循环不执行

可变关键字参数

- 示例：输出学生信息，除姓名外的其他信息项不固定

```
def show_info(name, **info):
    print(info)
    print("姓名: {}".format(name))
    for key, value in info.items():
        print("{}: {}".format(key, value))

show_info("张三", age=20, major="计算机")
show_info("李四")
```

运行结果：

{'age': 20, 'major': '计算机’ }
姓名：张三
age：20
major：计算机
{}
姓名：李四

强制位置参数与命名关键字参数

- 概念：
  - 强制位置参数：在参数列表中加入 /，/ 之前的形参只能按位置传入，不能使用“形参名=值”的形式（Python 3.8 起支持）
  - 命名关键字参数：在参数列表中加入 *，* 之后的形参只能使用“形参名=值”的形式传入，不能按位置传入
  - / 与 * 本身不是形参，只用于标记其前后形参的传入方式
  - 位于可变数量参数 *args 之后的形参，同样只能按名称传入

强制位置参数与命名关键字参数

- 概念：
  - 强制位置参数：在参数列表中加入 / ，/ 之前的形参只能按位置传入，不能使用“形参名=值”的形式（Python 3.8 起支持）
- 示例一：强制位置参数

```
def is_triangle(a, b, c, /):
    return a + b > c and b + c > a and a + c > b


print(is_triangle(3, 4, 5))
print(is_triangle(a=3, b=4, c=5))
```

运行结果：

True

TypeError: is_triangle() got some positional-only arguments passed as keyword arguments: 'a, b, c'

强制位置参数与命名关键字参数

- 概念：
  - 命名关键字参数：在参数列表中加入 *，* 之后的形参只能使用“形参名=值”的形式传入，不能按位置传入
- 示例二：命名关键字参数

```
def is_triangle(*, a, b, c):
    return a + b > c and b + c > a and a + c > b


print(is_triangle(a=3, b=4, c=5))
print(is_triangle(3, 4, 5))
```

运行结果：

True

TypeError: is_triangle() takes 0 positional arguments but 3 were given

强制位置参数与命名关键字参数

- 概念：
  - 位于可变数量参数 *args 之后的形参，同样只能按名称传入
- 示例三：*args 之后的形参

```
def show_total(name, *scores, unit="分"):
    total = 0
    for s in scores:
        total += s
    print("{}的总分为：{}{}".format(name, total, unit))

show_total("张三", 85, 90, 78)
show_total("李四", 4, 3, 5, unit="学分")
```

运行结果：
张三的总分为：253分
李四的总分为：12学分

⚠ scores 会接收其后所有的位置实参，因此 unit 只能按名称传入

默认值不宜使用可变对象

- 程序运行结果是什么？

```
def add_item(item, items=[]):
    items.append(item)
    return items


print(add_item("苹果"))
print(add_item("香蕉"))
print(add_item("橙子", ["葡萄"]))
print(add_item("梨"))
```

运行结果：

['苹果']
['苹果', '香蕉']
['葡萄', '橙子']
['苹果', '香蕉', '梨']

默认值不宜使用可变对象

- 概念：
  - 参数的默认值只在定义函数时创建一次，此后每次调用使用的都是同一个对象
  - 若默认值是列表、字典、集合等可变对象，且函数内修改了它的内容，修改结果会一直保留，并在此后每一次使用默认值的调用中不断累积
  - 正确做法：将默认值设为 None，在函数内判断后再创建新的对象

默认值不宜使用可变对象

- 程序运行结果是什么？

```
def add_item(item, items=[]):
    items.append(item)
    return items


print(add_item("苹果"))
print(add_item("香蕉"))
print(add_item("橙子", ["葡萄"]))
print(add_item("梨"))
```

第二次调用本应得到 ['香蕉']，但第一次调用添加的“苹果”仍保留在默认列表中，结果为 ['苹果', '香蕉']

运行结果：

['苹果']
['苹果', '香蕉']
['葡萄', '橙子']
['苹果', '香蕉', '梨']

默认值不宜使用可变对象

- 程序运行结果是什么？

```
def add_item(item, items=[]):
    items.append(item)
    return items


print(add_item("苹果"))
print(add_item("香蕉"))
print(add_item("橙子", ["葡萄"]))
print(add_item("梨"))
```

第四次调用再次使用默认列表，第一、二次添加的元素都依然存在，修改在多次调用之间累积

第三次调用传入了新列表，不使用默认值，因此不受影响

运行结果：

```
['苹果']
['苹果', '香蕉']
['葡萄', '橙子']
['苹果', '香蕉', '梨']
```

参数与返回值的类型标注

- 概念
  - 为形参和返回值标注预期的数据类型，用于说明函数应当如何使用，提高代码的可读性
  - 形参的类型写在形参名之后，用冒号分隔；返回值的类型写在参数列表之后、冒号之前，用 `->` 标注
  - 形参同时带有默认值时，类型标注写在默认值之前
  - ⚠ 类型标注只起说明作用，Python 解释器在运行时不检查实参的类型，传入其他类型的数据不会因此报错
- 语法格式

```
def 函数名(形参名: 类型, 形参名: 类型 = 默认值) -> 返回值类型:
    函数体
```

参数与返回值的类型标注

- 示例：根据及格线判断成绩是否及格

```
def is_pass(score: int, pass_score: int = 60) -> bool:
    return score >= pass_score


print(is_pass(75))
print(is_pass(52, pass_score=50))
print(is_pass(59.5))
```

⚠ 第三次调用传入浮点数 59.5，与标注的 int 不一致，但程序正常运行，表明类型标注不具有强制约束作用

运行结果：

```
True
True
False
```



2.2 用模块管理函数

命名冲突与模块

- 问题：命名冲突
  - 同一个程序中，若两个函数同名，后定义的函数会覆盖先定义的函数
- 示例：同一文件中定义两个同名函数

```
def area(r):
    return 3.14159 * r * r


def area(a, b):
    return a * b


print(area(3, 4))
print(area(2))
```

⚠ 第二个 area 覆盖了第一个，求圆面积的函数已不存在，调用 area(2) 时报错

运行结果：

```
12
TypeError: area() missing 1 required positional argument: 'b'
```

命名冲突与模块

- 解决方法：用模块管理函数
  - Python 中每一个 .py 文件就是一个模块，文件名（不含 .py）即为模块名
  - 不同模块中可以定义同名的函数，互不影响
  - 使用 import 模块名 导入模块后，以“模块名.函数名”的形式调用函数，即可区分不同模块中的同名函数
- 示例：将两个函数分别放入不同的模块

文件 circle.py：

```
def area(r):
    return 3.14159 * r * r
```

文件 rectangle.py：

```
def area(a, b):
    return a * b
```

命名冲突与模块

- 解决方法：用模块管理函数
  - Python 中每一个 .py 文件就是一个模块，文件名（不含 .py）即为模块名
  - 不同模块中可以定义同名的函数，互不影响
  - 使用 import 模块名 导入模块后，以“模块名.函数名”的形式调用函数，即可区分不同模块中的同名函数
- 示例：将两个函数分别放入不同的模块

```
import circle
import rectangle


print("{:.2f}".format(circle.area(2)))
print(rectangle.area(3, 4))
```

导入模块的方式

- 三种导入方式：

方式	语法	调用形式	特点
导入整个模块	`import 模块名`	`模块名.函数名()`	可区分不同模块的同名函数
导入模块中的函数	`from 模块名 import 函数名`	`函数名()`	书写简洁，但可能与其他同名函数冲突
使用别名	`import 模块名 as 别名` `from 模块名 import 函数名 as 别名`	`别名.函数名()` `别名()`	缩短较长的名称，或区分同名函数

- 示例一：为模块指定别名

```
import circle as c
import rectangle as rect

print("{:.2f}".format(c.area(2)))
print(rect.area(3, 4))
```

导入模块的方式

- 三种导入方式：

方式	语法	调用形式	特点
导入整个模块	`import 模块名`	`模块名.函数名()`	可区分不同模块的同名函数
导入模块中的函数	`from 模块名 import 函数名`	`函数名()`	书写简洁，但可能与其他同名函数冲突
使用别名	`import 模块名 as 别名` `from 模块名 import 函数名 as 别名`	`别名.函数名()` `别名()`	缩短较长的名称，或区分同名函数

- 示例二：从两个模块中导入同名函数

```
from circle import area
from rectangle import area

print(area(3, 4))
print(area(2))
```

⚠ 后导入的 rectangle 模块中的 area 覆盖了先导入的同名函数，命名冲突再次出现

导入模块的方式

- 三种导入方式：

方式	语法	调用形式	特点
导入整个模块	`import 模块名`	`模块名.函数名()`	可区分不同模块的同名函数
导入模块中的函数	`from 模块名 import 函数名`	`函数名()`	书写简洁，但可能与其他同名函数冲突
使用别名	`import 模块名 as 别名` `from 模块名 import 函数名 as 别名`	`别名.函数名()` `别名()`	缩短较长的名称，或区分同名函数

- 示例三：为导入的函数指定别名

```
from circle import area as circle_area
from rectangle import area as rect_area

print("{:.2f}".format(circle_area(2)))
print(rect_area(3, 4))
```



2.3 高阶函数

函数可以赋值给变量

- 概念：
  - 函数可以赋值给其他变量，通过新的变量名调用
  - 函数还可以作为实参传入另一个函数，也可以作为另一个函数的返回值
  - ⚠ 函数名后不加圆括号，表示函数本身；加圆括号，表示调用函数并得到其返回值

```
def square(x):
    return x * x
```

```
f = square
r = square(3)
```

将函数本身赋给 f，f 与 square 指向同一个函数，f(4) 等同于 square(4)

调用函数，将返回值 9 赋给 r，r 是整数

高阶函数

- 定义
  - 满足以下任意一个条件的函数，称为高阶函数：
    - 接收一个或多个函数作为参数
    - 将函数作为返回值
- 已经使用过的高阶函数：

函数	作为参数传入的函数
`sorted(序列, key=函数)`	`key` 参数接收一个函数，指定排序规则
`max(序列, key=函数)`	`key` 参数接收一个函数，指定比较规则
`min(序列, key=函数)`	`key` 参数接收一个函数，指定比较规则

高阶函数

- 示例：自定义一个高阶函数，对数据连续执行两次指定的操作

```
def square(x):
    return x * x

def apply_twice(func, x):
    return func(func(x))

print(apply_twice(square, 3))
print(apply_twice(lambda x: x + 10, 3))
```

⚠ apply_twice 的形参 func 接收一个函数，因此 apply_twice 是高阶函数

运行结果：

81
23

高阶函数

- 示例：通用运算函数
  - 利用下面的函数对任意多个数求和：

```
def total(*args):
    result = 0
    for item in args:
        result += item
    return result
```

  - 存在问题：
    - 函数体中写死了加法运算 +=，只能求和
    - 若还需要求乘积，就要再编写一个结构几乎相同、仅运算不同的函数，代码重复

高阶函数

- 示例：通用运算函数
  - 利用下面的函数对任意多个数求和：

```
def total(*args):
    result = 0
    for item in args:
        result += item
    return result
```

  - 解决思路：
    - 将“运算”本身作为参数传入：用一个函数表示运算，函数体中调用该函数，而不是直接使用 + 运算符

高阶函数

- 示例：通用运算函数

```
def calc(init_value, op_func, *args):
    result = init_value
    for item in args:
        result = op_func(result, item)
    return result

def add(x, y):
    return x + y

def mul(x, y):
    return x * y

print(calc(0, add, 1, 2, 3, 4, 5))
print(calc(1, mul, 1, 2, 3, 4, 5))
print(calc(0, lambda x, y: x + y * y, 1, 2, 3))
```

op_func 接收一个有两个参数的函数，表示运算规则

lambda 函数，计算平方和

运行结果：

```
15
120
14
```

内置高阶函数：map()

- 概念：
  - map() 将一个函数依次作用于序列中的每一个元素，得到由各元素的处理结果组成的新序列，这一操作称为映射
  - map() 的返回值是一个 map 对象，不能直接显示其中的元素，通常用 list() 将其转换为列表
  - 原序列不会被修改
- 语法格式
  - map(函数，序列)

内置高阶函数：map()

- 示例：将一组摄氏温度转换为华氏温度：

```
def to_fahrenheit(c):
    return c * 9 / 5 + 32


temps = [0, 25, 37, 100]
result = map(to_fahrenheit, temps)
print(result)
print(list(result))
print(list(map(lambda c: c * 9 / 5 + 32, temps)))
print(list(map(str, temps)))
```

用 list() 转换后，得到每个温度的转换结果

可用 lambda 函数代替单独定义的函数

传入内置函数 str，将每个整数转换为字符串

运行结果：

```
<map object at 0x7f6e0709aad0>
[32.0, 77.0, 98.6, 212.0]
[32.0, 77.0, 98.6, 212.0]
['0', '25', '37', '100']
```

内置高阶函数：filter()

- 概念：
  - filter() 将一个函数依次作用于序列中的每一个元素，保留函数返回值为 True 的元素，舍弃返回值为 False 的元素，这一操作称为过滤
  - 传入 filter() 的函数应当返回布尔值，用于表示“是否保留该元素”
  - 与 map() 相同，filter() 返回一个 filter 对象，通常用 list() 将其转换为列表；原序列不会被修改
- 语法格式
  - filter(函数，序列)

内置高阶函数：filter()

- 示例：从一组成绩中筛选出及格和不及格的成绩：

```
def is_pass(score):
    return score >= 60


scores = [85, 52, 90, 47, 76]
result = filter(is_pass, scores)
print(list(result))
print(list(filter(lambda s: s < 60, scores)))
print(scores)
```

用 lambda 函数指定相反的条件，筛选出不及格的成绩

运行结果：

```
[85, 90, 76]
[52, 47]
[85, 52, 90, 47, 76]
```

map() 与 filter() 的比较

比较项	`map()`	`filter()`
作用	对每个元素进行转换	按条件选取元素
传入函数的返回值	转换后的新值	布尔值
结果中的元素个数	与原序列相同	小于或等于原序列

map() 与 filter() 的组合使用

- 概念：
  - filter() 和 map() 的结果仍是可以遍历的序列，因此可以将一个函数的结果作为另一个函数的输入，组合完成“先筛选、再转换”的处理
  - 组合时，先执行内层的函数，再执行外层的函数
  - 同样的处理也可以用已学过的列表生成式完成
- 示例：去掉列表中的奇数，再求每个偶数的平方

```
nums = [35, 12, 8, 99, 60, 52]

evens = filter(lambda x: x % 2 == 0, nums)
print(list(map(lambda x: x ** 2, evens)))

print(list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, nums))))

print([x ** 2 for x in nums if x % 2 == 0])
```

第一种写法：分两步完成

第二种写法：一行完成，执行顺序为由内向外

第三种写法：列表生成式

嵌套函数与函数作为返回值

- 概念：
  - 嵌套函数：在一个函数的函数体内部定义另一个函数，内部的函数称为嵌套函数
  - 嵌套函数属于外层函数的局部内容，只能在外层函数内部使用，在外部无法直接调用
  - 嵌套函数可以读取外层函数的形参和局部变量
  - 外层函数可以将嵌套函数作为返回值返回；返回时只写函数名，不加圆括号。接收到的函数可以在外部调用，并且仍能使用外层函数调用时传入的参数值

嵌套函数与函数作为返回值

- 示例：生成“乘以指定倍数”的函数

make_multiplier 是高阶函数：它的返回值是一个函数

```
def make_multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))
print(triple(5))
print(type(double))
print(multiply(5))
```

读取了外层函数的形参 n

返回函数 multiply 本身

multiply 是嵌套函数，在外部无法直接调用，因此报错

运行结果：

```
10
15
<class 'function'>
NameError: name 'multiply' is not defined
```

偏函数

- 概念：
  - 偏函数：将一个函数的某些参数固定为指定的值，从而得到一个新函数；调用新函数时，无需再传入这些已固定的参数
  - 在 Python 中，使用标准库 functools 模块的 partial() 函数创建偏函数
  - partial() 接收一个函数作为参数，并返回一个新函数，因此它也是高阶函数
  - 调用偏函数时，仍可以用关键字参数的形式重新指定已固定的参数
- 语法格式：
  - functools.partial(原函数, 要固定的参数)

偏函数

- 示例一：固定内置函数 int() 的进制参数

```
import functools

int2 = functools.partial(int, base=2)
int16 = functools.partial(int, base=16)
```

int2 固定了 base=2

int() 默认将字符串按十进制转换为整数

```
print(int("1001"))
print(int("1001", base=2))
print(int2("1001"))
print(int16("1001"))
print(int2("1001", base=10))
```

运行结果：

```
1001
9
9
4097
1001
```

调用 int2("1001") 等同于 int("1001", base=2)

调用 int2 时重新指定 base=10，覆盖了固定值，按十进制转换

偏函数

- 示例二：由求幂函数生成求平方、求立方的函数

```
import functools

def power(x, n):
    return x ** n

square = functools.partial(power, n=2)
cube = functools.partial(power, n=3)

print(square(5))
print(cube(5))
```

运行结果：

```
25
125
```



2.4 装饰器

装饰器

- 问题引入
  - 现有下载文件和上传文件两个函数，需要统计每次调用所花费的时间

```
import time


def download(filename):
    print("开始下载{}".format(filename))
    time.sleep(1)
    print("{}下载完成".format(filename))


def upload(filename):
    print("开始上传{}".format(filename))
    time.sleep(2)
    print("{}上传完成".format(filename))
```

time.sleep(秒数) 使程序暂停指定的时间，此处用于模拟下载和上传所需的时间

装饰器

- 问题引入
  - 现有下载文件和上传文件两个函数，需要统计每次调用所花费的时间

```
start = time.time()
download("课程讲义.pdf")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))


start = time.time()
upload("实验报告.docx")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))
```

time.time() 返回当前时刻，单位为秒；在调用前后各记录一次，两者之差即为所花费的时间

装饰器

- 问题引入
  - 现有下载文件和上传文件两个函数，需要统计每次调用所花费的时间

```
start = time.time()
download("课程讲义.pdf")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))

start = time.time()
upload("实验报告.docx")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))
```

问题一：记录时间、计算并输出耗时的代码在每次调用时都要重复书写

装饰器

- 问题引入
  - 现有下载文件和上传文件两个函数，需要统计每次调用所花费的时间

```
start = time.time()
download("课程讲义.pdf")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))

start = time.time()
upload("实验报告.docx")
end = time.time()
print("花费时间: {:.2f}秒".format(end - start))
```

问题二：若在函数内部添加计时代码，又需要逐个修改原有函数，且计时功能与下载、上传本身的功能无关

装饰器的定义与结构

- 概念
  - 装饰器是一个函数，用于在不修改原函数代码的前提下，为原函数增加额外的功能
  - 装饰器接收被装饰的函数作为参数，返回一个增加了新功能的函数，因此装饰器是高阶函数
  - 装饰器内部通常定义一个嵌套函数（一般命名为 wrapper），其中依次完成三件事：执行新增功能的代码、调用原函数、返回原函数的返回值
  - wrapper 的参数写作 *args, **kwargs，可以接收任意形式的实参，并原样传给原函数，使装饰器适用于参数不同的各种函数

装饰器的定义与结构

- 基本结构

原函数

```
def 装饰器名(func):
    def wrapper(*args, **kwargs):
        # 调用原函数之前执行的代码
        result = func(*args, **kwargs)
        # 调用原函数之后执行的代码
        return result
    return wrapper
```

装饰器示例：编写计时装饰器 record_time

```python
import time

def record_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("{}执行时间: {:.2f}秒".format(func.__name__, end - start))
        return result
    return wrapper
```

func.__name__ 用于获取函数的名称，以便在输出中区分不同的函数

原函数不变

```python
def download(filename):
    print("开始下载{}".format(filename))
    time.sleep(1)
    print("{}下载完成".format(filename))

def upload(filename):
    print("开始上传{}".format(filename))
    time.sleep(2)
    print("{}上传完成".format(filename))
```

```python
download = record_time(download)
upload = record_time(upload)
```

将原函数传入装饰器，用返回的 wrapper 替换原来的 download

```python
download("课程讲义.pdf")
upload("实验报告.docx")
```

装饰器的@语法

- 概念
  - Python 为装饰器提供了更简洁的写法：在定义函数的 def 语句上方一行写 @装饰器名
  - 这种写法与“函数名 = 装饰器名(函数名)”的效果完全相同，但更直观：阅读函数定义时即可看出该函数被哪个装饰器装饰
  - 这类不增加新功能、只使代码书写更简便的语法，称为语法糖

装饰器的@语法

- 两种写法的对应关系：

直接调用装饰器	使用 @ 语法

`def download(filename):`
`    ...`
 	
`download = record_time(download)`
	`@record_time`
	`def download(filename):`
	`    ...`

装饰器的@语法

- 示例：

```
import time

def record_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("{}执行时间: {:.2f}秒".format(func.__name__, end - start))
        return result
    return wrapper

@record_time
def download(filename):
    print("开始下载{}".format(filename))
    time.sleep(1)
    print("{}下载完成".format(filename))
```

使用 functools.wraps

- 前提
  - 实际使用装饰器时，装饰后的函数通常会替换原来的同名函数，即采用以下两种等价写法之一：

```
download = record_time(download)
```

```
@record_time
def download(filename):
    ...
```

- 存在问题
  - 执行后，名称 download 不再指向原函数，而是指向装饰器返回的 wrapper 函数。由此会产生两个问题

使用 functools.wraps

- 两个问题：
  - 函数的名称等信息发生改变。装饰后得到的是 wrapper 函数，其名称为 wrapper，而不是原函数的名称
  - 无法调用未被装饰的原函数。原名称已指向 wrapper，原函数不再与任何名称关联
    - ⚠ 若写成 download1 = record_time(download)，名称 download 仍指向原函数，此时可以直接调用原函数，第二个问题不会出现

使用 functools.wraps

- 示例：两个问题

```
import time

def record_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("{}执行时间: {:.2f}秒".format(func.__name__, end - start))
        return result
    return wrapper

@record_time
def download(filename):
    print("开始下载{}".format(filename))
    time.sleep(1)
    print("{}下载完成".format(filename))

print(download.__name__)
download("课程讲义.pdf")
```

第一行输出：函数名称显示为 wrapper，而不是 download，体现第一个问题

第二行输出：调用 download(...) 时输出了执行时间，说明无法执行不计时的原函数，体现第二个问题

运行结果：

```
wrapper
开始下载课程讲义.pdf 课程讲义.pdf
下载完成
download执行时间：1.00秒
```

使用 functools.wraps

- 解决方法：
  - 使用标准库 functools 模块中的 wraps，它本身也是一个装饰器，写在 wrapper 函数的定义上方，形式为 @wraps(func)
  - 使用后，被装饰的函数保留原函数的名称等信息
  - 同时，原函数被记录在被装饰函数的 __wrapped__ 属性中，通过它可以调用未被装饰的原函数

使用 functools.wraps

- 示例：使用wraps 解决两个问题

```
import time
from functools import wraps

def record_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("{}执行时间: {:.2f}秒".format(func.__name__, end - start))
        return result
    return wrapper

@record_time
def download(filename):
    print("开始下载{}".format(filename))
    time.sleep(1)
    print("{}下载完成".format(filename))

print(download.__name__)
download("课程讲义.pdf")
download.__wrapped__("实验报告.docx")
```

- download.__name__ 恢复为download
- 调用 download(...) 时计时功能生效
- 调用 download.__wrapped__(...)时执行的是原函数，没有输出执行时间

运行结果：

```
download
开始下载课程讲义.pdf
课程讲义.pdf下载完成
download执行时间：1.00秒
开始下载实验报告.docx
实验报告.docx下载完成
```



2.5 进阶篇练习题

进阶篇练习题

- 1. 写出下列程序的运行结果

```
def f(a, **kw):
    return len(kw)

print(f(1, b=2, c=3))
```

答案：
2

1 按位置传给 a；b=2、c=3 由 **kw 接收，组装为字典 {'b': 2, 'c': 3}
字典中有 2 个键值对，len(kw) 为 2

进阶篇练习题

- 2. 写出下列程序的运行结果

```
def f(a, *, b):
    return a + b

print(f(1, b=2))
print(f(b=5, a=1))
print(f(1, 2))
```

答案：
3
6
TypeError: f() takes 1 positional argument but 2 were given

b 位于 * 之后，是命名关键字参数，只能按名称传入
第二次调用：两个参数均按名称传入，与书写顺序无关

进阶篇练习题

- 3. 写出下列程序的运行结果

```
def add(x, lst=[]):
    lst.append(x)
    return lst

add(1)
add(2)
print(add(3))
```

答案：
[1, 2, 3]

默认值 lst=[] 只在定义函数时创建一次，三次调用都未传入 lst，使用的是同一个列表

进阶篇练习题

- 4. 模块 ma.py 和 mb.py 中都定义了函数 hello()，分别返回“A” 和 “B”。下列程序的运行结果是：

```
from ma import hello
from mb import hello

print(hello())
```

答案：
B

两条 from ... import 语句导入了同名函数 hello，后导入的 mb 模块中的 hello 覆盖了先导入的同名函数

进阶篇练习题

- 5. 写出下列程序的运行结果

```
nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda x: x * 10, filter(lambda x: x % 3 == 0, nums))))
```

答案：
[30, 60]

由内向外执行：filter() 先保留能被 3 整除的元素，得到 3、6
map() 再将每个元素乘以 10，得到 30、60

进阶篇练习题

- 6. 写出下列程序的运行结果

```
from functools import partial

def power(x, n):
    return x ** n

cube = partial(power, n=3)
print(cube(2), cube(2, n=2))
```

答案：
8 4

cube 固定了 n=3，cube(2) 等同于 power(2, n=3)
cube(2, n=2) 中重新指定了 n=2，覆盖固定值，等同于 power(2, n=2)

进阶篇练习题

- 7. 写出下列程序的运行结果

```
def deco(func):
    def wrapper(*args):
        print("开始")
        func(*args)
    return wrapper

@deco
def double(x):
    return x * 2

print(double(4))
```

答案：
开始
None

调用 double(4) 实际执行 wrapper(4)
wrapper 先输出“开始”，再调用原函数 func(4)；原函数返回 8
wrapper 中没有 return 语句，因此返回 None，print() 输出 None

2.6 综合练习

综合练习

- 编写一个简易的购物车程序，实现以下功能：
  1. 向购物车中添加商品，记录商品名称、单价和数量，数量默认为 1
  2. 结算购物车的总金额，可以指定折扣；为避免误传，折扣只能以“名称=值”的形式传入
  3. 为会员提供专用的结算方式，固定享受八折优惠
  4. 每次结算时，自动输出“开始结算”和“结算完成”的提示信息，但不修改结算函数本身的代码
- 数据表示：
  - 购物车用一个列表表示，列表中的每个元素是一件商品
  - 每件商品用一个字典表示，包含 name（名称）、price（单价）、quantity（数量）三个键

综合练习

- 功能 1：添加商品
  - 要求： 记录商品名称、单价和数量，数量默认为 1

```
def add_item(cart, name, price, quantity=1):     # quantity 默认值为 1
    # 每件商品用字典表示，追加到购物车列表中
    cart.append({"name": name, "price": price, "quantity": quantity})

cart = []                                         # 购物车用列表表示
add_item(cart, "T恤", 99, 2)
add_item(cart, "帆布鞋", 199)                     # 未传入数量，使用默认值 1
add_item(cart, "笔记本", 15, 3)
print(cart)
```

综合练习

- 功能 2：结算总金额
  - 要求： 计算购物车的总金额，可以指定折扣；折扣只能以“名称=值”的形式传入

```
def checkout(cart, *, discount=1.0):     # * 之后的 discount 只能按名称传入，默认不打折
    # map() 对每件商品计算“单价 × 数量”，得到各商品的小计
    subtotals = map(lambda item: item["price"] * item["quantity"], cart)
    # 各商品小计求和后，乘以折扣
    return sum(subtotals) * discount

print(checkout(cart))
print(checkout(cart, discount=0.9))
```

综合练习

- 功能 3：会员结算
  - 要求： 为会员提供专用的结算方式，固定享受八折优惠

```python
from functools import partial

# 固定 checkout 的 discount 参数为 0.8，得到会员专用的结算函数
vip_checkout = partial(checkout, discount=0.8)

print(vip_checkout(cart))                    # 只需传入购物车，无需再指定折扣
```

综合练习

- 功能 4：输出结算提示
  - 要求： 每次结算时，自动输出“开始结算”和“结算完成”的提示信息，但不修改结算函数本身的代码

```
from functools import wraps

def log_checkout(func):
    @wraps(func)                                    # 保留原函数的名称等信息
    def wrapper(*args, **kwargs):                   # 接收任意形式的实参
        print("开始结算")                           # 调用原函数之前执行
        result = func(*args, **kwargs)              # 调用原函数，保存其返回值
        print("结算完成，应付金额：{:.2f}元".format(result))    # 调用原函数之后执行
        return result                               # 返回原函数的返回值
    return wrapper

@log_checkout                                       # 为 checkout 添加结算提示
def checkout(cart, *, discount=1.0):
    subtotals = map(lambda item: item["price"] * item["quantity"], cart)
    return sum(subtotals) * discount
```